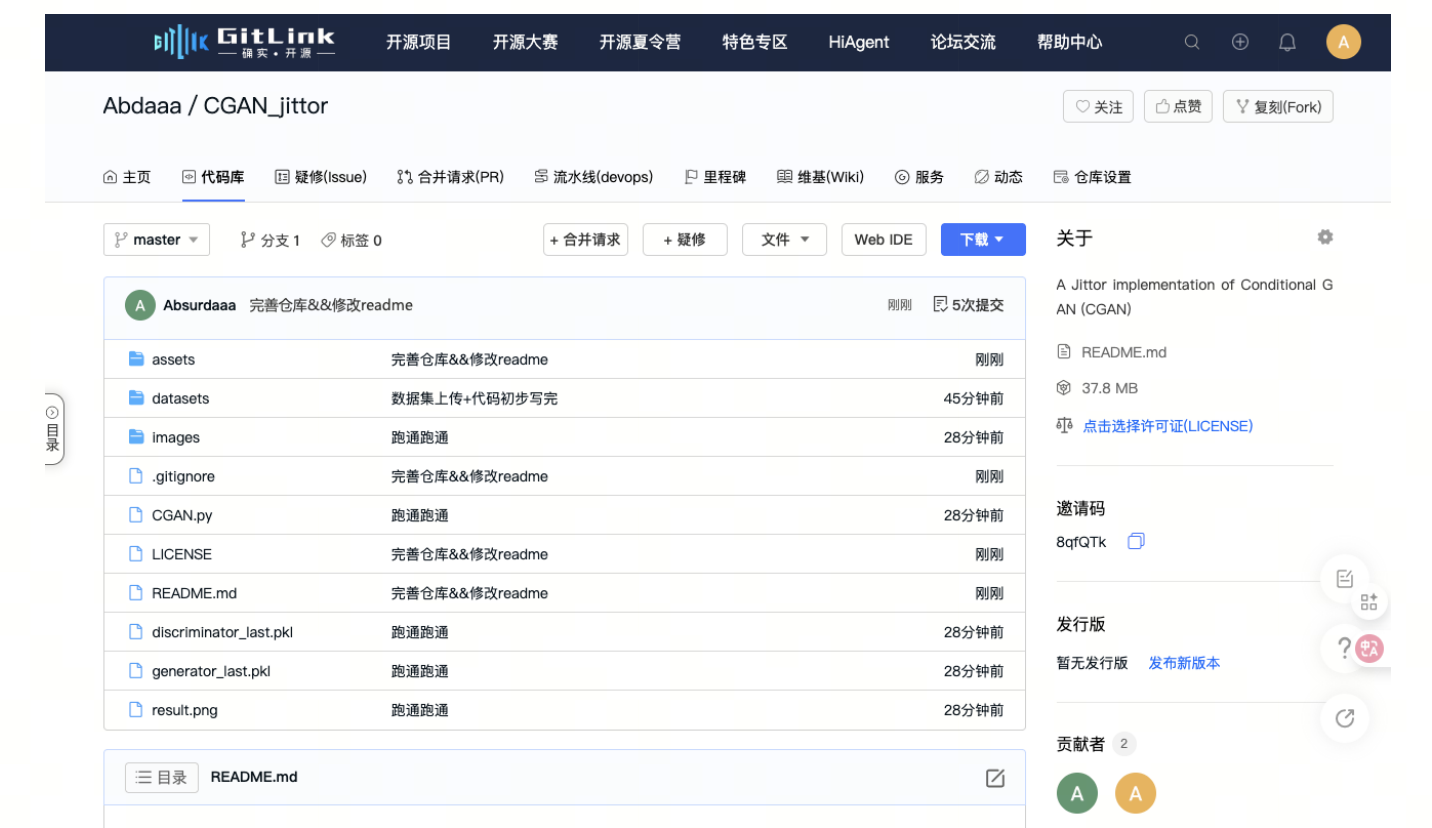


CGAN-计图热身赛实验报告

学院：计算机学院 学号：2312966 姓名：林晖鹏

一、项目开源

Gitlink项目开源地址：https://gitlink.org.cn/Abdaaa/CGAN_jittor



项目主页截图

二、环境配置

本次作业的主要难点主要在环境配置，由于本地没有gpu，我尝试在实验室的服务器上面配置，下面这个是参考了官方的安装指南：<https://cg.cs.tsinghua.edu.cn/jittor/download/>

```
代码块
1  conda create -n jittor-env python=3.7 -y
2  # 激活环境
3  conda activate jittor-env
4  # 安装计图
```

```
5 python -m pip install jittor
6 # 验证安装
7 python3.7 -m jittor.test.test_example
8 python3.7 -m jittor.test.test_cudnn_op
```

在配置过程中，由于服务器上的cuda为12.1，这个版本下缺少了CUSPARSE的链接库，而在服务器上面担心扰乱sudo的配置环境，也不好直接sudo下载。按照报错信息，我们用计图自带的配置代码，下载合适版本的用户级的cuda：`python3.7 -m jittor_utils.install_cuda`

代码块

```
1 RuntimeError: [f 1219 11:48:04.106759 00 compile_extern.py:251] CUDA found
  but cusparse is not loaded:
2 CUSPARSE library is not loaded,
3 please ensure it is installed along with the CUDA toolkit.
4 or you can let jittor install cuda and cudnn for you:
5 >>> python3.7 -m jittor_utils.install_cuda
```

配置完成之后，验证完毕：

```
-----
Ran 1 test in 0.347s
OK
```

```
-----
Ran 5 tests in 1.291s
OK
```

三、实现流程

GAN原理是本实验的背景+基础，这里不再赘述，详情可以阅读论文：

<https://arxiv.org/abs/1411.1784>

(1) 数据集下载

由于服务器机器无法连接外网，代码中的dataloader自动下载会失败，这里我们手动下载之后放到目录下的 `datasets/MNIST` 文件夹，并修改代码中的数据集构建的代码：

代码块

```
1 # 导入MNIST数据集
2 import jittor.transform as transform
3 transform = transform.Compose([
4     transform.Resize(opt.img_size),
```

```

5     transform.Gray(),
6     transform.ImageNormalize(mean=[0.5], std=[0.5]),
7 ])
8 # dataloader = MNIST(train=True,
9   transform=transform).set_attrs(batch_size=opt.batch_size, shuffle=True)
10 # 新--导入MNIST数据集
11 import jittor.dataset as datasets
12 mnist_path = "./datasets/MNIST/"
13 dataloader = datasets.MNIST(
14     train=True,
15     transform=transform,
16     data_root=mnist_path, # 指向存放4个.gz文件的目录
17     download=False # 强制禁用自动下载, 避免尝试联网
18 ).set_attrs(batch_size=opt.batch_size, shuffle=True, num_workers=opt.n_cpu) #
    对应你的opt参数

```

(2) 代码实现

主要实现三个地方的TODO

1. 添加最后一个线性层，最终输出为一个实数

为model添加最后一层线性层，观察代码前面部分，可以知道此时维度为512

代码块

```

1 self.model = nn.Sequential(nn.Linear((opt.n_classes +
2   int(np.prod(img_shape))), 512),
3   nn.LeakyReLU(0.2),
4   nn.Linear(512, 512),
5   nn.Dropout(0.4),
6   nn.LeakyReLU(0.2),
7   nn.Linear(512, 512),
8   nn.Dropout(0.4),
9   nn.LeakyReLU(0.2),
10  # TODO: 添加最后一个线性层，最终输出为一个实数
11  nn.Linear(512, 1)
12 )

```

2. 将d_in输入到模型中并返回计算结果

d_in输入模型中，调用model函数。

代码块

```
1 def execute(self, img, labels):
2     d_in = jt.contrib.concat((img.view((img.shape[0], (- 1))),
3     self.label_embedding(labels)), dim=1)
4     validity = self.model(d_in)
5     return validity
```

3. 计算损失函数

代码块

```
1 validity_real = discriminator(real_imgs, labels)
2 d_real_loss = adversarial_loss(validity_real, valid) # TODO: 计算真实类别的损失函数
3
4 validity_fake = discriminator(gen_imgs.stop_grad(), gen_labels)
5 d_fake_loss = adversarial_loss(validity_fake, fake) # TODO: 计算虚假类别的损失函数
```

(3) 实验结果

最后将ID改为本人学号

代码块

```
1 #TODO: 写入比赛页面中指定的数字序列 (字符串类型)
2 number = "2312966"
```

运行代码，可以得到结果图：

